

DESAFIO DO SABER: UMA APLICAÇÃO GERENCIADA SOB O REGIME DA GERÊNCIA DE CONFIGURAÇÃO E MUDANÇAS (GCM)

Vera Célia Gadelha
Ufersa
Pau dos Ferros, Brasil
vera.gadelha@alunos.ufersa.edu.br

Abstract—A Gerência de Configuração e Mudanças (GCM) é crucial para a estabilidade e a evolução sustentável de sistemas de *software*. Este artigo apresenta o Desafio do Saber, uma aplicação de quiz desenvolvida em Java, cuja arquitetura foi intencionalmente projetada sob os pilares da GCM. O sistema é composto por cinco Itens de Configuração (IC) de Lógica e Apresentação (*Questao*, *Jogador*, *GerenciadorDeQuestoes*, *Quiz* e *Demo*). A validação automatizada de cada *build* é assegurada pela camada de testes de unidade, o *QuizTeste.java*, que funciona como a ferramenta de auditoria de GCM. Esta arquitetura de baixo acoplamento garante que as mudanças (no conteúdo, lógica ou UI) sejam implementadas de forma rastreável, segura e de baixo risco, oferecendo um produto final robusto e manutenível [1], [10].

Palavras-chave—gerência de configuração, controle de mudanças, Java, arquitetura de software, testes de unidade, Clean Architecture.

I. INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

A Gerência de Configuração e Mudanças (GCM) é a disciplina que lida com a identificação, o controle, o *status* e a auditoria dos componentes do *software* ao longo do tempo [10]. Em ambientes de desenvolvimento contínuo, a ausência de controle sobre a evolução do código e dos dados leva a sistemas instáveis, difíceis de manter e caros de atualizar.

A. O Papel da GCM no Ciclo de Vida do Software

A GCM não é apenas um repositório de código (como o Git); ela engloba processos, ferramentas e políticas para gerenciar a evolução do produto de forma disciplinada [4], [10]. Seu objetivo central é garantir a integridade da Linha de Base (LB) e permitir que as Requisições de Mudança (RM) sejam implementadas e auditadas de maneira segura. A disciplina de GCM atua como a espinha dorsal de qualquer processo de desenvolvimento ágil ou tradicional [14].

B. O Contexto Java e Orientação a Objetos (OO)

A modularidade inerente ao Java facilita a identificação e o isolamento dos ICs (classes e interfaces), que são os alvos primários de controle de versão e auditoria [15]. A OO, através de princípios como Encapsulamento e Abstração, apoia a GCM ao definir fronteiras claras entre os Itens de Configuração (IC) [5].

Em aplicações dinâmicas como plataformas de quiz, o conteúdo (perguntas), a lógica de pontuação e a interface estão em constante evolução. O desafio principal é isolar estas áreas para garantir que uma modificação em uma camada (e.g., UI) não cause falhas na lógica de negócio, e vice-versa [3], [12].

O *Desafio do Saber* foi desenvolvido em Java para atender a esse requisito. A principal contribuição deste trabalho é demonstrar como a arquitetura de baixo acoplamento e a integração de testes de unidade são elementos vitais para implementar a GCM na prática, permitindo a evolução segura e auditável do produto [14].

II. ARQUITETURA DO SISTEMA E IMPLEMENTAÇÃO

O *Desafio do Saber* adota um modelo de separação de responsabilidades estrita, crucial para a Gerência de Configuração [6], [15]. O sistema é modelado em cinco Itens de Configuração (IC) primários, ilustrando o princípio de que cada componente deve ter uma única responsabilidade [5].

A. Itens de Configuração (IC) Primários

A Tabela I detalha o papel de cada IC e sua importância estratégica sob a ótica da GCM.

TABLE I
DETALHAMENTO DOS ITENS DE CONFIGURAÇÃO

IC	Classe	Responsabilidade Principal	Importância na GCM
IC1	Questao.java	Estrutura e conteúdo da pergunta.	Configuração de Conteúdo. Permite o versionamento e a adição de novos <i>datasets</i> .
IC2	Jogador.java	Rastreamento do desempenho e regras de <i>scoring</i> .	Lógica de Negócio. Isola e facilita alterações na política de pontuação.
IC3	GerenciadorDeQuestoes.java	Gerenciamento do conjunto de questões e categorias.	Fonte de Dados de Configuração. Ponto único de gerenciamento das Linhas de Base de conteúdo.
IC4	Quiz.java	Coordenação da sessão (seleção, embaralhamento, avanço).	Orquestração. Gerencia o ciclo de vida do quiz e transição de estado.
IC5	Demo.java	Interface Gráfica (Swing).	Separação da Camada UI. Mudanças na aparência não quebram a lógica.

B. Baixo Acoplamento e Manutenibilidade

A arquitetura do sistema é caracterizada pelo baixo acoplamento [1]. A classe *Quiz* (IC4) apenas consome serviços de outros ICs: ela não armazena o estado do jogador (IC2) nem renderiza a interface (IC5).

C. Exemplo de Implementação e Baixo Acoplamento

A implementação da classe *Demo* (IC5) apenas interage com a interface pública dos ICs de Lógica (e.g., *Quiz.iniciarNovaSessao()* e *Jogador.getPontuacao()*). Se houver uma requisição de mudança para migrar de Swing para uma aplicação Web ou terminal, o código dos ICs 1, 2, 3 e 4 permanecerá intocado, reduzindo o custo e o risco da mudança tecnológica [3], [11]. Essa independência funcional é a realização prática do baixo acoplamento, sendo um dos pilares da GCM e da Arquitetura Limpa.

III. GCM E GARANTIA DE QUALIDADE

O desenvolvimento do *Desafio do Saber* foi gerenciado com um processo de GCM que se baseia no controle de Linhas de Base e na auditoria contínua através de testes.

A. Controle de Versão e Linhas de Base

O conceito de Linha de Base (LB) é usado para identificar e controlar versões estáveis do produto [4], [10].

1) *Fluxo de GCM e Progressão do Projeto*: A progressão do projeto ilustrou o fluxo de GCM:

2) *Linha de Base Inicial*: Linha de Base 1 (LB-1): Produto inicial com as categorias Português e Matemática.

3) *Requisição de Mudança*: Requisição de Mudança (RM-001): Foi solicitada a expansão do escopo com a adição da categoria Cidadania (30 novas questões). Esta RM exigiu alterações rastreadas nos ICs: IC3 (Inventário) e IC5 (Apresentação).

4) *Linha de Base Final*: Linha de Base 2 (LB-2): O *release* final para produção, após a implementação bem-sucedida e validação da RM-001.

Este processo garante que cada *release* do produto seja uma configuração conhecida e rastreável, permitindo *rollback* seguro para versões anteriores, se necessário [14].

B. Auditoria e Qualidade (O Papel do QuizTeste)

A classe *QuizTeste.java* é o componente de Auditoria de Configuração e Garantia de Qualidade do projeto [9], [13]. Ela é fundamental para validar a integridade do sistema após qualquer modificação em um dos ICs de lógica.

1) *Detalhes da Auditoria*: Validação da Lógica: Testes como *deveIncrementarPontuacaoSeRespostaCorreta* validam a configuração da regra de negócio (IC2 - Jogador).

Integração Contínua (CI): A existência de testes de unidade é o primeiro passo para a implementação de um sistema de CI/CD [2], [8]. Em um *pipeline* de CI, a execução automatizada do *QuizTeste* em cada *commit* garante que a auditoria de *build* seja um processo contínuo e obrigatório, reduzindo drasticamente o risco de defeitos em produção.

Resultados da Auditoria de Configuração: O sucesso na integração contínua e no processo de *build* mais recente confirmou a eficácia desta abordagem. A execução automatizada do *QuizTeste.java* foi concluída com êxito, validando a integridade da Linha de Base atual (LB-2) e atestando a conformidade dos Itens de Configuração após a implementação da RM-001, o que formaliza a conclusão da etapa de Auditoria de Configuração.

IV. ANÁLISE DE MÉTRICAS E VISUALIZAÇÃO DO FLUXO GCM

A arquitetura do *Desafio do Saber* foi projetada visando métricas de qualidade chave, sendo as principais Coesão e Acoplamento, que têm impacto direto na manutenibilidade e no sucesso da GCM [4].

A. Métricas de Qualidade de Software

1) *Coesão e Acoplamento*: Coesão: A Coesão mede o quão relacionadas e focadas estão as responsabilidades dentro de um módulo (classe) [1]. Idealmente, a Coesão deve ser alta. O IC1 (*Questao*) demonstra alta coesão, pois é responsável apenas por dados e sua manipulação.

Acoplamento: O Acoplamento mede o grau de dependência entre os módulos [1]. Idealmente, o Acoplamento deve ser baixo. O sistema atinge baixo acoplamento pois os ICs de lógica (*Questao*, *Jogador*) não dependem da interface (*Demo*), isolando a lógica da apresentação e garantindo que as mudanças na UI não exijam mudanças no coração do sistema.

B. Visualização do Fluxo de Gerência de Mudanças

Para ilustrar o processo disciplinado de GCM do projeto, o Fluxo de Mudança Controlada é mapeado abaixo:

1. Identificação da Necessidade (RM-001): Requisição de mudança para adicionar a categoria "Cidadania". 2. Análise do Impacto: Determinar que a RM-001 impacta o IC3 (Inventário/Dados) e o IC5 (Apresentação). 3. Implementação e

Teste Local: O código é alterado e o desenvolvedor executa o *QuizTeste.java* localmente para garantir a integridade da regra de negócio (IC2). 4. Validação da Linha de Base: Após o *commit*, a ferramenta de CI/CD (simulada pelo *build* final) executa a Auditoria Automatizada (*QuizTeste.java*). 5. Promoção da Linha de Base: Se todos os testes passarem, a Configuração é promovida para a Linha de Base 2 (LB-2), pronta para o *release*.

Este fluxo garante que o custo de corrigir defeitos descobertos após a LB ser estabelecida é minimizado, um princípio chave da GCM.

V. FERRAMENTAS DE GCM E CONTROLE DE VERSÃO

O controle de versão é o primeiro e mais fundamental pilar de qualquer sistema de GCM [7]. O projeto *Desafio do Saber* utiliza o Git como ferramenta de controle de versão, aplicando uma estratégia de ramificação (*branching strategy*) que suporta o isolamento de mudanças.

A. O Uso Estratégico do Git

O Git atua como o repositório central dos Itens de Configuração (ICs), permitindo o rastreamento histórico de cada mudança (Quem, Quando, Porquê).

1) *Rastreabilidade da Mudança*: Cada Requisição de Mudança (RM), como a RM-001, é implementada em uma *branch* isolada (e.g., *feature/rm-001-cidadania*). O *commit* final que promove esta *branch* para a *main* (ou Linha de Base) serve como o registro formal da mudança, atrelado ao código-fonte, garantindo a rastreabilidade completa, conforme exigido pela GCM [7].

B. Integração Contínua (CI) e o Build Auditado

A filosofia de Integração Contínua (CI) é essencial para a Auditoria de Configuração. A cada *merge* ou *push* de código, um processo automatizado de *build* é acionado [8], [2].

1) *Automação da Auditoria*: O *QuizTeste.java* é o centro dessa automação. Ao invés de uma auditoria manual demorada, o sistema de CI executa os testes de unidade. A Linha de Base (LB) só pode ser atualizada (promovida) se o *build* for bem-sucedido e todos os testes passarem. Este processo garante que o produto final seja sempre consistente e livre de regressões conhecidas, atendendo ao requisito de estabilidade da GCM [2].

VI. O PRINCÍPIO DE ISOLAMENTO E A ARQUITETURA LIMPA (CLEAN ARCHITECTURE)

O sucesso da GCM no *Desafio do Saber* é uma consequência direta da adesão a princípios arquiteturais modernos, como os propostos pela Arquitetura Limpa [3]. Este modelo enfatiza a separação estrita de camadas, onde as regras de negócio centrais (ICs 1, 2, 3 e 4) são isoladas das interfaces (IC5) e *frameworks* [3].

A. Lei da Dependência e Estabilidade

A Arquitetura Limpa obedece à Lei da Dependência, que dita que as dependências do código devem apontar sempre para dentro. As camadas externas (e.g., a UI - IC5, que é mais volátil) devem depender das camadas internas (e.g., a lógica de pontuação - IC2, que é mais estável), e nunca o contrário.

1) *Lógica de Negócio (ICs Estáveis)*: Os Itens de Configuração IC1 (*Questao*) e IC2 (*Jogador*) representam as Entidades e Regras de Negócio do sistema. Sua estabilidade é máxima, pois raramente precisam de alteração.

2) *Apresentação (IC Volátil)*: O Item de Configuração IC5 (*Demo*) representa o Frameworks & Drivers (a camada externa/UI). Ele é volátil e pode ser descartado e substituído (e.g., migração para web) sem quebrar o código interno dos ICs estáveis.

O isolamento proporcionado por essa arquitetura facilita a GCM de forma radical: uma Requisição de Mudança (RM) na interface (*alterar cores do Swing*) só afeta o IC5, exigindo apenas o re-versionamento e a re-auditoria deste único item, sem impactar a Linha de Base dos ICs de Lógica.

B. Gerenciamento de Dependências

A forma como os ICs se comunicam é crucial. O uso de Interfaces (ou Abstrações em Java) para a comunicação entre camadas, em vez de implementações concretas (o princípio da Inversão de Dependência), permite que a orquestração (IC4) utilize o serviço de dados (IC3) sem saber se o IC3 está lendo dados de um arquivo ou de um repositório interno. Essa abstração é a chave para a manutenibilidade a longo prazo e o gerenciamento de versões [5].

VII. METODOLOGIAS ÁGEIS E A GCM (DEVOPS)

A GCM é frequentemente vista como um processo estático de documentação. No entanto, em ambientes ágeis e de DevOps, ela se transforma em um sistema dinâmico de controle automatizado [8].

A. GCM como Habilitador do DevOps

DevOps é a união de desenvolvimento e operações. A GCM é o mecanismo que garante que o *release* contínuo seja seguro e confiável.

1) *Definindo Linhas de Base no DevOps*: Build Artifacts: O JAR compilado do *Desafio do Saber* é um Item de Configuração de Liberação formalmente rastreado pelo número da *build* (e.g., *desafio-saber-lb-2.1.0*).

Implantação Automatizada: O sucesso da Auditoria Automatizada (*QuizTeste.java*) atua como o *gate* de qualidade que aciona a implantação na próxima fase (e.g., testes de aceitação ou produção), garantindo que apenas configurações validadas se tornem Linhas de Base de produção [2].

B. Métricas de Processo GCM no Contexto de DevOps

Para medir a eficácia do processo de GCM, são utilizadas métricas de *throughput* e estabilidade [8].

1) *Taxa de Sucesso do Build*: Taxa de Sucesso do Build: A alta taxa de sucesso do *build* após a implementação de RMs indica que a arquitetura isolada (ICs) está protegendo o sistema contra quebras de integração.

2) *Tempo Médio para Reparo (MTTR)*: Tempo Médio para Reparo (MTTR): O MTTR é o tempo necessário para restaurar o serviço após um erro. A GCM, ao definir Linhas de Base conhecidas e rastreadas, permite um *rollback* imediato e seguro para a última LB estável, reduzindo o MTTR drasticamente em comparação com sistemas sem controle de configuração [7].

VIII. CONCLUSÃO

O *Desafio do Saber* é um sistema Java exemplar para a disciplina de GCM. O projeto validou o modelo de separação de responsabilidades por meio de seus cinco Itens de Configuração de baixo acoplamento [1], [15]. A análise das métricas de coesão e acoplamento, em conjunto com o fluxo de GCM disciplinado (envolvendo RMs, Linhas de Base e testes), reforça a qualidade da arquitetura. *O sucesso da integração contínua e dos testes de unidade comprova a robustez e a manutenibilidade do sistema sob o regime da GCM.*

A adesão aos princípios da Arquitetura Limpa isolou a lógica de negócio (ICs 1, 2, 3, 4) da tecnologia de apresentação (IC5), tornando o sistema resistente à obsolescência tecnológica e ideal para o gerenciamento de mudanças. A aplicação prática de princípios de Engenharia de Software e o uso de ferramentas de CI/CD resultaram em uma configuração estável e prontas para futuras expansões, como a externalização de dados de configuração e a migração de UI, ambas controladas pela GCM.

IX. TRABALHOS FUTUROS

Para a próxima fase de evolução e o aperfeiçoamento da GCM do *Desafio do Saber*, propõe-se:

A. Externalização de Dados

1) *Refatoração do IC3*: Refatoração do IC3: Refatorar o IC3 (*GerenciadorDeQuestoes*) para ler as questões a partir de arquivos de configuração externos (JSON ou XML), convertendo o conteúdo de código-fonte para um artefato de configuração rastreado separadamente [7]. Isso transformará o conteúdo de um IC de código para um IC de dado, permitindo que a GCM gerencie o conteúdo do quiz de forma mais ágil.

B. Desenvolvimento de Nova UI (Mudança Controlada)

1) *Prova de Robustez da GCM*: Prova de Robustez da GCM: Como prova da robustez da GCM, desenvolver uma nova interface de usuário (e.g., Web ou terminal) para o sistema. Esta mudança controlada apenas exigirá a criação de um novo IC de Apresentação (Novo IC5), reutilizando completamente os ICs de Lógica e Dados (IC1, IC2, IC3, IC4), comprovando a eficácia da arquitetura modular [1], [3].

REFERENCES

- [1] Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley Professional.
- [2] Humble, J., & Farley, D. (2020). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.
- [3] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [4] Leite, J. C. S. P., & Oliveira, A. C. M. (2014). *Engenharia de Software: Foco na Qualidade e nas Métricas*. LTC.
- [5] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (2021). *Design Patterns: Elements of Reusable Object-Oriented Software* (25th Anniversary Edition). Addison-Wesley Professional.
- [6] Parnas, D. L. (2017). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 60(2), 29-37.
- [7] Kersten, T. (2023). *Practical GitOps: Continuous Delivery and Configuration Management*. Apress.
- [8] Kim, G., Debois, P., & Willis, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations*. IT Revolution Press.
- [9] Kiefel, K. (2021). *Unit Testing Principles, Practices, and Patterns*. Manning Publications.
- [10] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- [11] Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley Professional.
- [12] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- [13] Seemann, M. (2014). *Dependency Injection Principles, Practices, and Patterns*. Manning Publications.
- [14] Sutherland, J. (2014). *Scrum: The Art of Doing Twice the Work in Half the Time*. Crown Business.
- [15] Chen, L., & Baresi, L. (2020). *Software Architecture: Foundations, Theory, and Practice*. John Wiley & Sons.